

Chapter 4

Advanced Thread Synchronization

5 Lecture · 영상 87 분 · Ubuntu Server 26.04 + C
Observe → Understand → Implement → Analyze

L16 Race Condition 을 CPU 관점에서 분석하기

L17 Atomic Operation 과 Compare-And-Swap

L18 CPU Memory Ordering 과 Memory Barrier

L19 Futex 내부 구조

L20 Lock Contention 과 Lock-Free Programming 입문

L16

Race Condition 을 CPU 관점에서 분석하기

Chapter 4. Advanced Thread Synchronization
counter++ 는 한 줄이지만 CPU 에서는 세 단계다

영상 16 분

▶ 애니메이션 [C4-1 Race Condition Interleaving](#)

문제 상황

```
$ ./race_counter
expected 200000000
actual    13121383    (lost 6878617)
$ ./race_counter
actual    10344699    (lost 9655301)
```

- 실행할 때마다 다르다
항상 **작은** 쪽으로만 틀린다

Chapter 4 의 질문

Lecture	질문
L16	보호하지 않으면 CPU 안에서 무슨 일이 일어나는가
L17	Lock 없이 안전한 연산은 어떻게 가능한가
L18	Compiler 와 CPU 가 순서를 바꾼다는 것은
L19	mutex 는 언제 Kernel 에 들어가는가
L20	Lock 이 있는데 왜 느린가

`counter++`의 기계어

```
mov    0x2dea(%rip),%rax    # Load : Memory → Register
add     $0x1,%rax           # Add   : Register 에서 +1
mov     %rax,0x2ddf(%rip)    # Store : Register → Memory
```

- **Register**는 Thread 마다 따로
- **Memory**는 공유

각 Thread는 공유 값을 자기 복사본으로 가져가 계산하고, 그 결과로 **덮어쓴다**

→ Terminal 전환: `objdump -d race_counter`

두 Thread 의 Interleaving

화면 전환 → [animation.html](#)

순차 실행 → Lost Update → 오래된 값으로 덮어쓰기 → lock add

▶ 애니메이션 [C4-1 Race Condition Interleaving](#)

CPU 가 하나여도 발생한다

```
$ taskset -c 0 ./race_counter
actual    10344699    (lost 9655301)
```

	원인
CPU 여러 개	두 Core 가 진짜 동시에 실행
CPU 하나	Load 와 Store 사이에서 Preemption

문제의 본질은 " 동시 실행 " 이 아니라 " 쪼개질 수 있는 연산 "

-O2 에서는 " 고쳐진 " 것처럼 보인다

```
$ gcc -O2 ... && ./race_counter_02  
actual      200000000    (lost 0)
```

```
add    %rax,0x2c79(%rip)      # counter += loops  ( 한 번 )
```

- Loop 가 사라졌다 . Race 의 기회가 2 천만 번 → 2 번

Data Race = **Undefined Behavior**. Compiler 는 " 다른 Thread 는 없다 " 고 가정한다

- " 돌려 보니 맞더라 " 는 증거가 아니다

Data Race vs Race Condition

```
if (stock > 0) {      /* check */  
    stock--;          /* act   */  
}
```

```
buyer 1: 구매 성공  
buyer 2: 구매 성공  
sold = 2, stock = -1
```

Data Race	보호 없는 동시 Memory 접근
Race Condition	Timing 에 따라 결과가 달라지는 논리 오류

보호 단위는 변수가 아니라 논리적 동작 전체

ThreadSanitizer 와 mutex

```
WARNING: ThreadSanitizer: data race
  Read of size 8 ... by thread T1:    race_counter.c:16
  Previous write ... by thread T2:    race_counter.c:16
  Location is global 'counter'
```

gcc -fsanitize=thread · 5~15 배 느림 · 실행된 경로만 검출

```
$ ./race_mutex          actual 200000000    ( 훨씬 느리다 )
```

→ Terminal 전환

정리

counter++ = Load / Add / Store. 사이에 끼어들면 Lost Update

- CPU 하나에서도 Preemption 으로 발생
- Data Race 는 UB. 최적화에 따라 증상이 숨는다
- Data Race $\neq$ Race Condition
- ThreadSanitizer

다음 : L17. 세 단계를 한 명령으로

L17

Atomic Operation 과 Compare-And-Swap

모든 동기화 도구의 바닥에 있는 CPU 명령

영상 17 분

▶ 애니메이션 [C4-2 Compare-And-Swap Simulator](#)

문제 상황

- 증가 하나를 위해 mutex 는 과하다
- Load-Add-Store 를 **쪼개지지 않게** 할 수 없을까

Atomic Operation: 다른 Thread 가 보기에 " 전혀 실행되지 않았거나 , 완전히 끝났거나 ". 중간 상태가 관찰되지 않는다

atomic_fetch_add → `lock addq`

```
static atomic_long counter;  
atomic_fetch_add(&counter, 1);
```

```
f0 48 83 05 36 2d 00    lock addq $0x1,0x2d36(%rip)    # <counter>
```

mov / add / mov 세 명령 → **한 명령** . f0 = lock Prefix

→ *Terminal 전환*

`lock` 은 무엇을 잠그는가

Core 0: lock addq \$1, counter

- ① counter 의 Cache Line 을 독점 확보 → 다른 Core 의 복사본 무효화
- ② 읽기 → +1 → 쓰기 (다른 Core 의 접근은 대기)
- ③ 해제

명령 하나여도 CPU 내부에서는 읽기 - 계산 - 쓰기 → 다른 **Core** 가 끼어들 수 있다

lock = 명령 실행 동안 **Cache Line** 독점

- 공짜가 아니다 → L20

Compare-And-Swap

```
Atomic {  
    if (*obj == *expected) { *obj = desired;    return true;  }  
    else                      { *expected = *obj; return false; }  
}
```

" 아직 내가 본 값 그대로라면 새 값으로 바꿔라 . 아니면 실패를 알려라 "

x86-64: lock cmpxchg · _weak 는 Loop 안에서 사용

Compare-And-Swap Simulator

화면 전환 → [animation.html](#)

경쟁 없음 → 경쟁 (실패와 재시도) → ABA

▶ 애니메이션 [C4-2 Compare-And-Swap Simulator](#)

CAS Loop Pattern

```
old = atomic_load(&counter);  
while (!atomic_compare_exchange_weak(&counter, &old, old + 1))  
    retries++;
```

/* 1. 읽는다 */
/* 2. 계산 3. CAS */
/* 실패 → 다시 */

actual 100000000

CAS 재시도 1284377 회

← L16 이었다면 Lost Update 었을 횟수

mutex = 비관적 (먼저 잠근다) / CAS = 낙관적 (해 보고 충돌했으면 다시)

→ Terminal 전환

CAS 로 Spin Lock 만들기

```
void spin_lock(void)    { while (atomic_flag_test_and_set(&f)) __builtin_ia32_pause(); }  
void spin_unlock(void) { atomic_flag_clear(&f); }
```

```
pause  
xchg    %al, lock_flag
```

- 기존 과정에서 개념으로 배운 Spin Lock 의 실체
- 기다리는 동안 CPU 100%. Lock 보유자가 Preemption 되면 모두 헛돈다 → L19 futex

비교, 그리고 ABA

pthread_mutex	612 ms
atomic_fetch_add	95 ms
CAS loop	240 ms

Atomic 은 변수 하나의 단일 연산만 보호한다

```
top → [A] → [B]      T1: old=A, next=B 읽고 멈춤
T2: pop A, pop B, free(B), push A
T1: CAS(top, A→B) 성공! → top 이 해제된 B 를 가리킨다
```

CAS 는 값만 본다 . Version Tag, Hazard Pointer, Epoch

정리

Atomic = 중간 상태가 보이지 않는 연산 . x86: lock Prefix = Cache Line 독점

atomic_fetch_add → lock add

CAS = lock cmpxchg. 읽기 → 계산 → CAS → 실패하면 재시도

- Atomic 명령 하나로 Spin Lock. 모든 Lock 의 바닥
- ABA. Atomic 은 단일 변수만

다음 : L18. atomic_store 와 $x = 1$ 은 둘 다 mov 처럼 보이는데 왜 다른가

L18

CPU Memory Ordering과 Memory Barrier

Source 에 적은 순서대로 실행된다는 보장은 없다

영상 18 분

▶ 애니메이션 [C4-3 Store Buffer와 Memory Ordering](#)

문제 상황 : 멈추지 않는 Thread

```
static int stop;  
void *worker(void *a) { while (!stop) spins++; ... }  
/* main */ sleep(1); stop = 1; pthread_join(...);
```

main : stop = 1 로 설정 , worker 를 기다립니다 ...
Alarm clock

stop 은 분명히 1 이 되었다 . int 하나의 쓰기라서 Lost Update 도 아니다

→ Terminal 전환

Compiler가 한 일

```
mov    0x2d32(%rip),%eax    # stop 을 한 번 읽는다
test   %eax,%eax
jne    12f0
jmp     12ee                # 12ee: 자기 자신으로 점프 = 무한 Loop
```

```
if (!stop) for (;;) ;      /* Compiler 가 바꾼 코드 */
```

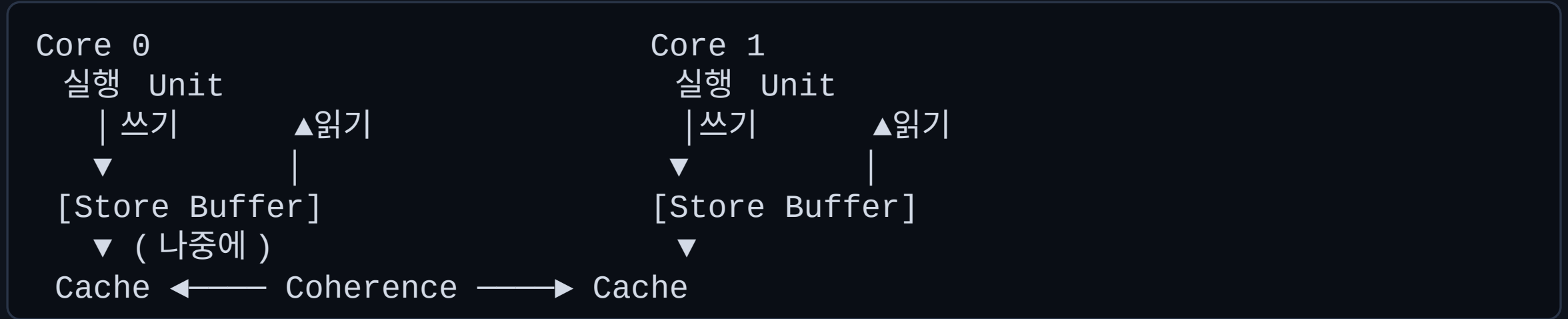
"Loop 안에서 stop 을 바꾸는 코드가 없다 → 한 번만 읽자 " · -00 에서는 동작한다

atomic 으로 고치기 . volatile 은 ?

	<code>volatile</code>	<code>atomic_*</code>
읽기 / 쓰기 생략 방지	O	O
Atomic 보장	X (v++ 는 여전히 3 단계)	O
CPU 수준의 순서 보장	X	O
C 표준상 Data Race	여전히 UB	아님

`volatile` 은 Memory-Mapped I/O 와 Signal Handler 를 위한 것

이번에는 CPU: Store Buffer



쓰기는 Store Buffer에 넣고 바로 다음 명령으로

- 자기 읽기는 Buffer를 먼저 본다 → Single Thread는 차이를 못 느낀다
- 다른 Core는 Buffer 안을 볼 수 없다

불가능해 보이는 결과

Thread 1 (CPU 0)	Thread 2 (CPU 1)
x = 1;	y = 1;
r1 = y;	r2 = x;

(r1, r2) = (0, 1)	48211	
(r1, r2) = (1, 0)	51402	
(r1, r2) = (1, 1)	241	
(r1, r2) = (0, 0)	146	<- 순차적으로는 불가능

Store-Load Reordering · Dekker / Peterson Algorithm 이 실제 CPU 에서 깨지는 이유

→ Terminal 전환

Store Buffer와 Memory Ordering

화면 전환 → [animation.html](#)

Store Buffer로 (0,0) → Barrier 적용 → Compiler의 읽기 제거

▶ **애니메이션** [C4-3 Store Buffer와 Memory Ordering](#)

Memory Barrier

```
$ ./store_buffer_fixed
(r1,r2) = (0,0)          0
    xchg    %eax, x      # atomic_store 가 mov 가 아니라 xchg
```

mfence, lock 명령 , xchg = Store Buffer 를 비우는 Full Barrier

C11 기본 Order memory_order_seq_cst → Store 뒤에 Barrier 필요

Kernel: smp_mb()(CPU 용) / READ_ONCE(), WRITE_ONCE()(Compiler 용)

x86 은 관대한 편 : TSO vs Weak Ordering

순서 바뀜	x86-64 (TSO)	ARM, RISC-V
Store-Load	가능	가능
Store-Store, Load-Load, Load-Store	없음	가능

```
data = 42;  ready = 1;           /* ARM: ready 가 먼저 보일 수 있다 */
```

실무 규칙 : 공유 변수는 ****mutex** 또는 **atomic_***. **mutex** 안에서는 이 문제를 생각하지 않아도

정리

- Compiler: 공유 변수의 읽기를 제거하거나 옮긴다
- CPU: Store Buffer → Store-Load Reordering

Memory Barrier: `mfence`, `lock`, `xchg`. `atomic_store(seq_cst) = xchg`

- x86 = TSO, ARM = Weak

`volatile` 은 동기화 도구가 아니다

다음 : L19. mutex 는 Spin 과 Sleep 을 어떻게 조합하는가

L19

Futex 내부 구조

pthread_mutex_lock() 은 System Call 이 아니다

영상 18 분

▶ 애니메이션 [C4-4 Futex: Fast Path와 Slow Path](#)

문제 상황 : 두 가지 나쁜 선택

	경쟁 없음	경쟁 있음
Spin Lock	빠름	CPU 100% 낭비
매번 Kernel 요청	매번 System Call	잠들어서 양보

mutex 는 대부분 경쟁 없이 획득된다 → 흔한 경우는 User Space 에서 , 드문 경우만 Kernel 에서

futex = Fast Userspace muTEX

경쟁 없는 mutex: System Call 0 회

```
$ strace -c -e trace=futex ./mutex_nocontend  
lock/unlock 1000000 회, 1 회당 15.0 ns  
  calls  syscall  
    0    total
```

pthread_mutex_t 는 User Space 의 구조체 . lock = 그 안의 int 에 대한 CAS (0 → 1)

→ *Terminal* 전환

경쟁하는 mutex: futex 가 나타난다

```
$ strace -f -c -e trace=futex ./mutex_contend 4
```

calls	errors	syscall
284512	61230	futex

```
futex(0x5591c2e04040, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
```

```
futex(0x5591c2e04040, FUTEX_WAKE_PRIVATE, 1) = 1
```

첫 인자 = **mutex** 변수의 주소 . Kernel 에는 "mutex 객체 " 가 없다

futex System Call

op	의미
FUTEX_WAIT	*uaddr 이 아직 val 과 같으면 재워라 . 다르면 EAGAIN
FUTEX_WAKE	uaddr 에서 자는 Thread 를 최대 val 개 깨워라

	담당
상태 (잠겼는가)	User Space 의 정수 + Atomic 연산
재우기 , 깨우기	Kernel . 값의 의미는 모른다

→ Code: *futex_basic.c*

WAIT 는 왜 값을 다시 확인하는가

A: state 확인 → 잠겨 있다 → " 자야겠다 "

B: unlock (state = 0), WAKE → 자는 Thread 없음

A: futex(WAIT) → 잠든다 → 영원히

Lost Wakeup = Check-Then-Act (L16)

Kernel 이 Lock 을 잡은 채 *uaddr == val 을 재확인 → 다르면 재우지 않고 EAGAIN

strace 의 errors = Bug 가 아니라 **Lost Wakeup** 을 막은 횟수

Fast Path 와 Slow Path

화면 전환 → [animation.html](#)

경쟁 없음 → 경쟁 → Lost Wakeup 방지

▶ 애니메이션 [C4-4 Futex: Fast Path와 Slow Path](#)

futex 로 mutex 만들기

state	의미
0	안 잠김
1	잠김 , 대기자 없음
2	잠김 , 대기자 있을 수 있음

```
threads 1    FUTEX_WAIT 0 회 ,      FUTEX_WAKE 0 회  
threads 4    FUTEX_WAIT 41873 회 , FUTEX_WAKE 45102 회
```

1 과 2 를 구분하는 이유 = **unlock 의 Fast Path**

→ Code + Terminal: *my_mutex.c*

Kernel 안에서 , 그리고 futex 위의 것들

FUTEX_WAIT: 주소 → Key → Hash Bucket (Spin Lock) → 값 재확인 → 대기 List → schedule()
FUTEX_WAKE: 같은 Bucket → Key 가 같은 Thread 를 Runnable 로

▪ 경쟁 전까지 Kernel 은 mutex 의 존재를 모른다 . 100 만 개를 만들어도 Kernel 자원 0
cond, sem, pthread_join, rwlock, barrier, C++/Go/Rust/Java 의 mutex → 모두
futex

strace -p 에서 FUTEX_WAIT 로 멈춰 있다 = Lock 대기 . 전부 그렇다면 Deadlock 의심

정리

- futex = 흔한 경우는 User Space, 드문 경우만 Kernel
- 상태는 User Space 의 정수 , Kernel 은 WAIT / WAKE 만
- WAIT 의 값 재확인 = Lost Wakeup 방지 (EAGAIN)
- 주소 기반 Hash Table. 경쟁 없으면 Kernel 자원 0
- 0 / 1 / 2 상태로 unlock 도 Fast Path

다음 : L20. Lock 이 올바른데 왜 느린가

L20

Lock Contention과 Lock-Free Programming 입문

Thread 를 늘렸는데 빨라지지 않는다

영상 18 분

▶ 애니메이션 [C4-5 Cache Line과 False Sharing](#)

문제 상황

```
$ ./false_sharing
```

```
1630 ms
```

```
$ ./false_sharing pad
```

```
410 ms
```

a, b 는 서로 다른 변수 . Lock 도 Race 도 없다

변수의 " 위치 " 만 바꿨다

threads	global	atomic	local
1	810	330	32
4	3120	720	9

Cache Line 과 MESI

CPU 는 Memory 를 **64Byte Cache Line** 단위로 가져온다

상태	의미	쓰려면
Modified	나만 , 수정함	바로
Exclusive	나만 , 수정 전	바로
Shared	다른 Core 도 가짐	상대를 무효화
Invalid	내 복사본 무효	가져와야 함

쓰려면 **Line** 을 독점해야 한다 . 단위는 변수가 아니라 Line

Cache Line Ping-Pong

화면 전환 → [animation.html](#)

같은 Line → Padding → True Sharing

▶ 애니메이션 [C4-5 Cache Line과 False Sharing](#)

False Sharing 해결

```
struct padded_pair { alignas(64) volatile long a; alignas(64) volatile long b; };
```

흔한 발생 위치

long count[NTHREADS]; 각 Thread가 count[id]++

구조체의 인접 Field (Queue의 head / tail)

pthread_mutex_t locks[N];

Native Linux: perf c2c · 모든 변수에 붙이면 오히려 손해

→ *Terminal 전환*

Thread 를 늘리면 빨라지는가

Mode	결과	이유
global mutex	느려진다	직렬화 + futex Sleep/Wakeup + Context Switch + Line 이동
atomic	빨라지지 않는다	lock add 마다 Line 독점 . Lock-Free ≠ Contention-Free
local	선형으로 빨라진다	공유하는 것이 없다

→ Terminal 전환: `./lock_scale 4`

Contention 을 줄이는 설계

더 빠른 Lock 을 찾기 전에 덜 공유하는 구조를 찾는다

방법	예
공유하지 않기	Thread 별 Data + 합산 , per-CPU 변수
Critical Section 줄이기	Lock 안에서 I/O, malloc 금지
Lock 쪼개기	Hash Bucket 별 Lock
읽기 / 쓰기 구분	rwlock, RCU
Batch	한 번 잠그고 여러 개

Lock-Free Stack

```
void push(struct node *n) {  
    struct node *old = atomic_load(&top);  
    do { n->next = old; } while (!atomic_compare_exchange_weak(&top, &old, n));  
}
```

```
pushed 1000000, popped 1000000  
sum expected 500000500000  
sum actual   500000500000
```

이 예제가 안전한 이유 : Node 를 해제도 재사용도 하지 않는다 (ABA 회피)

→ *Code + Terminal*

Lock-Free 의 의미와 선택 기준

	보장
Blocking	Lock 보유자가 멈추면 모두 멈춘다
Lock-Free	누군가는 항상 전진한다
Wait-Free	모두가 유한 단계 안에 끝난다

기본은 **mutex** (경쟁 없으면 십수 ns)

단일 변수는 **atomic**

느리면 공유를 줄인다

Lock-Free 는 검증된 **Library** 로

Chapter 4 정리

counter++ = Load / Add / Store	L16
Atomic 명령: lock add, lock cmpxchg, xchg	L17
Compiler 와 CPU 는 순서를 바꾼다 → atomic_*, Barrier	L18
mutex = Atomic CAS + futex (경쟁할 때만 Kernel)	L19
비용의 근원 = Cache Line 의 이동 → 공유를 줄여라	L20

Worksheet 20 문제 → Chapter 5: Linux Event Driven Programming

다음 시간 · Chapter 5

Linux Event Driven Programming

개념 워크시트

[worksheet.html](#)

Lecture 당 10 문항 · 50 문항

애니메이션

[animation.html](#)

5 종

강의 자료

[lecture.html](#)

빈칸 채우기 · 괄호 넣기

저장소 github.com/imguru-mooc/adv-sysprog · 온라인 자료 imguru-mooc.github.io/adv-sysprog